

Document 02 — APIs, HTTP, JSON & Backend Basics

This is for self-study and live mentoring. Full plan: [CURRICULUM.md](#)

Prerequisites

[01_python_foundations](#) — you'll reuse `config.py` and `logging_setup.py` here.

How to Read & Practice This Document

- **What:** talking to other services over the internet (HTTP), in a way that doesn't break under real-world conditions.
- **Why:** every call to an LLM is really just an HTTP call underneath. If you can't handle timeouts and retries here, you won't be able to handle them for OpenAI either — you'll just be confused one layer up instead of here.
- **When:** any time your code talks to something outside itself, which in AI projects is almost always.
- **How to practice:** 1. Read the material once, just to get the shape of it. 2. Do the **Basic** exercises closed-book. This shows what you actually remember. 3. Do the **Intermediate/Real-world/Edge case** exercises with notes open — that's normal. 4. Try the **Build Task** without looking at any old solution. Let yourself get stuck before asking for a hint. 5. Use **Hint 1** → **Hint 2** only after really trying. Ask for the full solution only if you say "**Show me the solution.**" 6. Before moving on, explain out loud what your retry code actually does, and why. If you can't, you're not done.

Jump to: [Core Concepts](#) · [Practice Exercises](#) · [Build Task](#)

The Story — what this document is actually building

Picture Doc01's config and logger already working. Now your program needs to actually go get something from the outside world — call OpenAI, call any API. That's what this whole document is about: the network is unreliable, and your code has to expect that, not be surprised by it.

First, every call is a **request** that gets a **response** — a status code tells you broadly what happened (worked, your fault, their fault), but it doesn't tell you *what to do next*. That's a decision you have to

make yourself, based on which kind of failure it was.

Second, a network call can just hang forever if you don't set a **timeout** — the server never replies, and your program never notices, it just waits. So every single outbound call gets a maximum wait time, no exceptions.

Third, some failures are worth trying again (a timeout, a busy server, a rate limit) and some aren't (a bad request is still bad the second time you send it). **Retrying with backoff** is the disciplined way to try again — wait a little longer each time, add some randomness, and give up after a fixed number of tries instead of looping forever.

Fourth, even a response that "worked" (status 200) can still be lying to you — the body might not be valid JSON at all, or it might be valid JSON missing the exact field your code expects. Trusting a 200 blindly is how confusing bugs happen three functions downstream from the actual problem.

That's the story: HTTP basics teach you what a request/response actually promises. Timeouts stop your program from hanging silently. Backoff-with-jitter is how you retry without making things worse. And checking JSON shape (not just parsing it) closes the gap between "the network worked" and "the data is actually what I expected." The **Build Task** at the end turns all four of these into one `http_client.py` that every later document — including Doc04's OpenAI calls — reuses instead of re-solving this from scratch.

Core Concepts (read this first — everything you need is here)

HTTP: a request gets a response

Every API call is a **request** (a method, a web address, some headers, maybe a body) that gets answered by a **response** (a status code, headers, a body). The method tells the server what you're trying to do: `GET` reads data without changing anything, `POST` creates or triggers something, `PUT / PATCH` update something, `DELETE` removes it. **Why this matters beyond trivia:** each method carries a promise about repeating it — calling `GET` or `PUT` twice should leave things the same as calling it once. `POST` usually doesn't promise that (call it twice, and you might get two results instead of one). This is exactly why it's safe to automatically retry a failed `GET`, but you need to think harder before retrying a `POST`. **How status codes work:** the first digit tells you the category — 2xx means it worked, 3xx means you got redirected, 4xx means *your* request was wrong (don't just retry the same way), 5xx means the *server* failed (safe to retry). 429 is the one exception inside the 4xx group: it means "you weren't wrong, you were just too fast" — it's safe to retry, usually with a `Retry-After` header telling you how long to wait.

Timeouts: the failure that never tells you it's happening

A network call with no timeout can hang **forever** if the server never replies — it doesn't fail loudly, it just sits there, holding up your program indefinitely. **Why this is especially risky for AI projects:** LLM calls are slower and fail more often than typical API calls — bigger requests, longer processing

time, shared servers under load — so "it'll probably come back eventually" is a false comfort that turns into a genuinely stuck program. **When to add a timeout:** on every single outbound call, no exceptions, starting with the very first line of code that touches the network. **How it works:** a timeout is just a maximum wait time you set yourself. Once it's reached, your own code raises an error instead of waiting on the server forever. Two numbers usually matter: the *connect* timeout (how long to wait to even connect) and the *read* timeout (how long to wait for data once connected) — libraries like `requests` let you set both.

Retrying with exponential backoff and jitter

The simplest kind of retry — "just try again right away" — actually makes things worse under load. If a server is already struggling, and every client that fails instantly retries, you've just multiplied the exact load that caused the problem in the first place. **Exponential backoff** waits longer and longer between attempts (like 1s, then 2s, then 4s, then 8s), giving a struggling server room to recover. **Jitter** adds a small random amount to that wait, so that many clients failing at the same moment (like a shared rate limit) don't all retry at exactly the same instant and collide again. **Why the 4xx/5xx difference matters here:** retrying a 5xx or a timeout makes sense — the server might recover. Retrying a 400 (a bad request) or a 401 (bad login info) is pointless — the request is wrong, and it'll stay wrong every time until *you* fix it, not the server. **Why a cap matters:** retry forever, and a genuinely broken connection turns into an infinite loop, wasting time (and, with paid APIs, money). Always retry up to a fixed limit, then give up with a clear error.

JSON: two different kinds of "wrong"

A JSON response can be wrong in two very different ways, and mixing them up causes confusing bugs. **(1) Invalid JSON** — the response isn't parseable JSON at all (a cut-off response, an HTML error page instead of JSON, an empty body). This fails right at `json.loads()` or `response.json()`, with a clear parsing error. **(2) Valid JSON, wrong shape** — it parses fine, but a piece of data your code expects (like `response["choices"][0]["message"]`) isn't there, or is a different type than you expected. This kind of error shows up *later*, often as a confusing `KeyError` or `TypeError` several lines away from the actual API call — which is exactly why it's better to check the shape right after parsing, instead of trusting it and finding out much further downstream. **Why this distinction matters:** your error handling should treat "the server sent garbage" and "the server sent something valid but unexpected" as two separate cases, handled separately.

Go Deeper (Optional)

You don't need any of these to understand the Core Concepts above — use them if you want a second explanation or more detail.

- [requests library docs \(Quickstart\)](#) — the HTTP client you'll use everywhere, before switching to async clients later.
- [MDN — HTTP response status codes](#) — know 200/201/400/401/403/404/429/500/503 by heart.

- [MDN — HTTP request methods](#) — GET/POST/PUT/PATCH/DELETE, and what "safe to repeat" means.
- [Python json module docs](#) — `loads / dumps` , and where each one can fail.

Practice Exercises

Setup for this document's practice code: work inside `02_apis_http_json/` (same venv as before — if it's not active, `source .venv/bin/activate`). New package for this document: `pip install requests` .

How to run each exercise: group your practice code by topic, not by difficulty level. If two exercises below are really about the same thing, save them together in ONE script named after that topic — for example, if two exercises are both about `.env` config, save both in one file like `env_config_practice.py` , with each level's version as its own clearly labeled section inside it. Run each topic's file directly, for example: `python retry_backoff_practice.py` .

For this document, save your practice code as: - **Basic** (your first real API call) is its own topic — save it as `api_first_call_practice.py` . - **Intermediate** (timeouts and retry-with-backoff) and **Failure** (respecting a real rate limit) are both about retrying a failed call correctly — save them together as `retry_backoff_practice.py` , one section per level. - **Real-world** (one reusable session instead of repeated headers) is its own topic — save it as `session_reuse_practice.py` . - **Edge cases** (200 doesn't mean safe to trust) is its own topic — save it as `response_validation_practice.py` .

Why group by topic instead of by level: if you save each exercise by difficulty level instead, the different versions of the same idea end up scattered across separate files, and you can never see how one topic grows from simple to harder in one place. Grouping by topic keeps that growth visible — open one file, and you see the whole journey for that one thing, from basic to advanced, side by side.

Jump to an exercise: [Basic](#) · [Intermediate](#) · [Real-world](#) · [Edge cases](#) · [Failure](#) · [Build Task](#)

Basic — your first real API call

- **What:** call any free public API with `requests` , print the status code and the parsed JSON, then deliberately break the URL and read the exact error type Python gives you.
- **Why:** you need to see, once, that a bad URL and a bad response are two completely different kinds of failure with different exception types — that distinction is the whole document.
- **When you'll hit this for real:** the very first time you wire up any external API call, in any project, ever.
- **How to code it:** `requests.get("https://api.github.com")` , print `.status_code` and `.json()` . Then change the URL to something malformed (like `"http://broken"`) and wrap the call in `try/except` — print the exception's type with `type(e).__name__` .

- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Intermediate — timeouts and retry-with-backoff

- **What:** add a timeout to a call, force it to actually fire, then wrap a call in a retry loop with exponential backoff and a limit on attempts.
- **Why:** this exact wrapper is what `02_apis_http_json`'s Build Task turns into `http_client.py` — every project after this one calls through it.
- **When you'll hit this for real:** any time an outside service (including OpenAI) is slow or briefly down — without this, your whole program just hangs or crashes instead of recovering.
- **How to code it:** `requests.get(url, timeout=(3, 5))` pointed at an address that won't respond (like `http://10.255.255.1`), catch `requests.Timeout`. Then write a `for attempt in range(max_attempts):` loop that retries with `time.sleep(2 ** attempt)` between tries.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Real-world — one reusable session instead of repeated headers

- **What:** wrap login info (a fake bearer token is fine) into one `requests.Session()` object, instead of passing headers to every call by hand.
- **Why:** repeating the same headers dict in five different functions is exactly how one of them ends up with a typo'd or missing header — a session object makes that impossible.
- **When you'll hit this for real:** any client class you build for a real API (including your own `http_client.py`) — you'll set the auth header once, in one place.
- **How to code it:** `session = requests.Session()`, `session.headers.update({"Authorization": "Bearer fake-token"})`, then make 2-3 calls through `session.get(...)` instead of `requests.get(...)` and confirm the header goes out on all of them.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Edge cases — 200 doesn't mean safe to trust

- **What:** handle a response with status 200 but a body that isn't valid JSON, and separately, a response with valid JSON but a field your code expects is missing.
- **Why:** a status code only tells you the *transport* succeeded — it says nothing about whether the *content* is what you expected. Mixing up these two is a very common real bug.
- **When you'll hit this for real:** an API having a bad day and returning an HTML error page with a 200 status, or an API changing its response shape without warning you.
- **How to code it:** fake a `Response`-like object (or point at a URL that returns HTML) and call `.json()` inside a `try/except json.JSONDecodeError`. Separately, parse a real JSON dict and access a key that isn't there with `.get("missing_key")` vs. `["missing_key"]` — see the difference in what happens.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Failure — respecting a real rate limit

- **What:** simulate a 429 response and handle it correctly — respecting a `Retry-After` header if the server sent one, falling back to exponential backoff if not.
- **Why:** OpenAI (and most real APIs) rate-limit you, and a client that doesn't back off correctly makes the problem worse for itself and everyone sharing that API key.
- **When you'll hit this for real:** the first time you run an agent loop (Doc07) fast enough to actually hit OpenAI's rate limit — this exact handling is what keeps that agent working instead of crashing.
- **How to code it:** build a fake response object with `status_code = 429` and a `headers = {"Retry-After": "2"} . Write a function that checks for that header, sleeps that long if present, or falls back to 2 ** attempt seconds if not.`
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Build Task — HTTP Client Wrapper

Stuck on the Build Task? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Goal: a small `http_client.py` that every later project will use for outbound calls.

Requirements:

- Wraps `requests` with a default timeout (never call without one).
- Retries temporary failures (timeouts, 5xx, 429) with exponential backoff and jitter, up to a limit.
- Does NOT retry permanent failures (any 4xx besides 429) — fails right away, with the response body attached to the error.
- Logs each attempt at `DEBUG` level, and each final failure at `ERROR` level (use the logger from `01_python_foundations`).

Inputs: a URL, an HTTP method, optional headers/body, optional limits on retries/timeout.

Outputs: the parsed JSON response, or a clear, typed error if it fails.

Constraints: no bare `except: .` Your retry code should be testable without making real network calls (design it so you can plug in a fake version).

Suggested files:

```
02_apis_http_json/
├─ http_client.py
├─ exceptions.py
├─ test_http_client.py
```

Functions/Components to build:

- `exceptions.py` → `TransientHTTPError`, `PermanentHTTPError`
- `http_client.py` → `request_with_retry(method, url, **kwargs)` -> dict
- a backoff helper, like `compute_backoff_delay(attempt: int) -> float`

Expected Behavior

- A timeout, a 5xx, or a 429 → retried up to the limit, then raises `TransientHTTPError`.
- A 400/401/403/404 → raises `PermanentHTTPError` right away, no retry.
- A 200 with bad JSON → raises a clear parsing error, not a raw, uncaught `JSONDecodeError`.

Test Cases

Scenario	Expected
Fake 200 + valid JSON	Returns the parsed dictionary
Fake 429, then 200 on retry	Returns the parsed dictionary after waiting, attempt is logged
Fake 404	<code>PermanentHTTPError</code> right away, zero retries
Fake timeout on every attempt	<code>TransientHTTPError</code> after the limit is reached
Fake 200 with invalid JSON	A clear parsing error, not an unhandled crash

Break-It / Debug Preview

- A retry loop with no limit (a retry storm that never ends).
- Backoff with no jitter, tested with many "callers" failing at once (they all retry together).
- Full debugging drill in [14_debugging_lab](#).

Interview Topics Preview

- Repeating a call safely (idempotency) · exponential backoff + jitter · why you never retry a 400 the same way · REST-style APIs · what a 429 requires a good client to do.

Move On When

Given just a stack trace, you can tell whether the failure is your bug or the server's. Full details: [CURRICULUM.md §4](#).

Stuck? Ask for **Hint 1** through **Hint 2**. Ask for the full solution only if you say "**Show me the solution.**"